PHINMA-COC

Carmen, Cagayan de Oro City

OFFICIAL FINAL PROJECT TECHNICAL DOCUMENTATION

Desktop-Based Ordering System for Cherry Cosmetics Shop

Technical System Manual, Deployment Guide & Defense Framework

Course: ITE-298 Information Management (Database Systems 1 and 2)

Submitted by:

Team Leader

John Carl S. Vanzuela

Members

Cherrygel P. Antivo

Chrysler John M. Roble

Elijah Shiene F. Canillo

Jevy G. Sultan

Course: ITE-298 Information Management (Database Systems 1 and 2)

Document Type: Technical System Manual, Deployment Guide & Defense Framework

1. System Overview

1.1 Primary Purpose

The Desktop-Based Ordering System for Cherry Cosmetics Shop is a Java desktop application developed to organize the shop's product, inventory, order, customer, payment, and sales records. Customers can create an account, browse products, add products to a cart, place orders, and monitor order status. The administrator can manage products and categories, view and process orders, manage customer records, record payment details, and generate sales reports.

The system aims to reduce reliance on scattered messages and handwritten records and make routine order processing easier to manage and monitor.

1.2 Tech Stack Architecture

The system uses Java technologies and a local MySQL database.

- **User Interface:** Java Swing provides the application's desktop screens, forms, buttons, tables, and dashboards for customer and administrator operations.
- **Application Logic:** Java handles application operations, including account validation, product management, cart calculations, order processing, inventory updates, payment records, and report generation, according to the implemented source code.
- **Database Management:** MySQL stores account information, product categories, product details, customer orders, order items, and payment records.
- **Database Administration:** phpMyAdmin provides a browser-based interface for managing the MySQL database when used with XAMPP.
- **Database Connection:** Java Database Connectivity (JDBC), together with MySQL Connector/J, allows the Java application to communicate with the MySQL database.
- **Development Environment:** NetBeans IDE and Apache Ant support project development, compilation, building, and execution.

1.3 Key System Features

- **Product Management (CRUD):** Allows the administrator to create, view, and update product records and archive products when the archive function is implemented. Product information may include product names, prices, discounts, images, categories, descriptions, and stock quantities.
- **Customer Account Management:** Supports customer registration, login, account information management, and order viewing according to the available application functions. Administrator access to customer records depends on the implemented administrative interface.
- **Ordering and Cart Management:** Allows customers to browse products, select quantities, add items to a cart, calculate order totals, and submit orders.

- **Inventory Management:** Maintains product stock information and checks availability during order processing. Stock deduction, cancellation handling, and stock restoration must follow the workflow implemented in the application.
- **Order and Payment Management:** Allows the administrator to review and process orders. Payment methods, payment statuses, reference numbers, and payment dates may be recorded in the database. The application records payment information; it does not automatically process actual online payments unless a separate payment integration has been implemented.
- **Sales Reporting and Data Export:** Supports sales reporting and CSV export if these functions are present in the current application. Weekly, monthly, and yearly filters must be verified against the actual reporting code.
- **Receipt Generation:** Supports PNG receipt generation for eligible orders if the receipt-generation function is implemented and connected to the order-processing workflow.

2. Database Design & Schema Normalization (3NF)

2.1 Schema Entities & Attributes

The database consists of six (6) main tables that store related information and help keep records organized.

- **USERS** (id PK, username UNIQUE, email UNIQUE, password, role, is_archived)
  Stores administrator and customer accounts and identifies their roles.
- **CATEGORIES** (id PK, name UNIQUE, is_archived)
  Stores product category names.
- **PRODUCTS** (id PK, name, category_id FK, price, stock, discount, description, image, is_archived)
  Stores cosmetic product details, prices, discounts, descriptions, images, and available stock.
- **ORDERS** (id PK, customer_id FK, order_date, location, contact, status, total, return_reason, received_date)
  Stores customer order information, delivery details, order totals, and order status.
- **ORDER_ITEMS** (id PK, order_id FK, product_id FK, qty, unit_price)
  Stores the products, quantities, and prices associated with each order.
- **PAYMENTS** (id PK, order_id UNIQUE FK, method, status, reference, payment_date)
  Stores payment methods, payment statuses, reference numbers, and payment dates. The unique order_id allows at most one payment record per order.

2.2 Normalization Breakdown (1NF to 3NF)

- **First Normal Form (1NF):** Each table should have a primary key, and each field should contain a single value rather than a list of repeating values. For example, products belonging to an order are stored in separate ORDER_ITEMS records instead of being stored together in one field.
- **Second Normal Form (2NF):** A table should satisfy 1NF, and every non-key attribute should depend on the whole candidate key. In ORDER_ITEMS, the quantity and recorded unit price describe the product entry in a particular order. The main order information is stored separately in ORDERS.
- **Third Normal Form (3NF):** A table should satisfy 2NF and avoid non-key attributes depending on other non-key attributes. Account information, product categories, product details, order

information, order items, and payment records are separated into their respective tables to reduce unnecessary duplication and improve data consistency.

2.3 Crow's Foot ERD Cardinality & Relationships

- **CATEGORIES to PRODUCTS (One-to-Many):** One category can be associated with multiple products. Each product references a category through category_id, if the relationship is mandatory in the actual schema.
- **USERS to ORDERS (One-to-Many):** One customer can place multiple orders. Each order references a user through customer_id.
- **ORDERS to ORDER_ITEMS (One-to-Many):** One order can contain multiple order-item records. Each order item belongs to one order.
- **PRODUCTS to ORDER_ITEMS (One-to-Many):** One product can appear in multiple order-item records. Each order item references one product.
- **ORDERS to PRODUCTS (Many-to-Many):** An order can contain different products, and a product can appear in different orders. The ORDER_ITEMS table resolves this many-to-many relationship.
- **ORDERS to PAYMENTS (One-to-Zero-or-One):** An order can have zero or one payment record when PAYMENTS.order_id is unique and the database permits an order to exist without a payment record. Each payment record must reference one existing order.

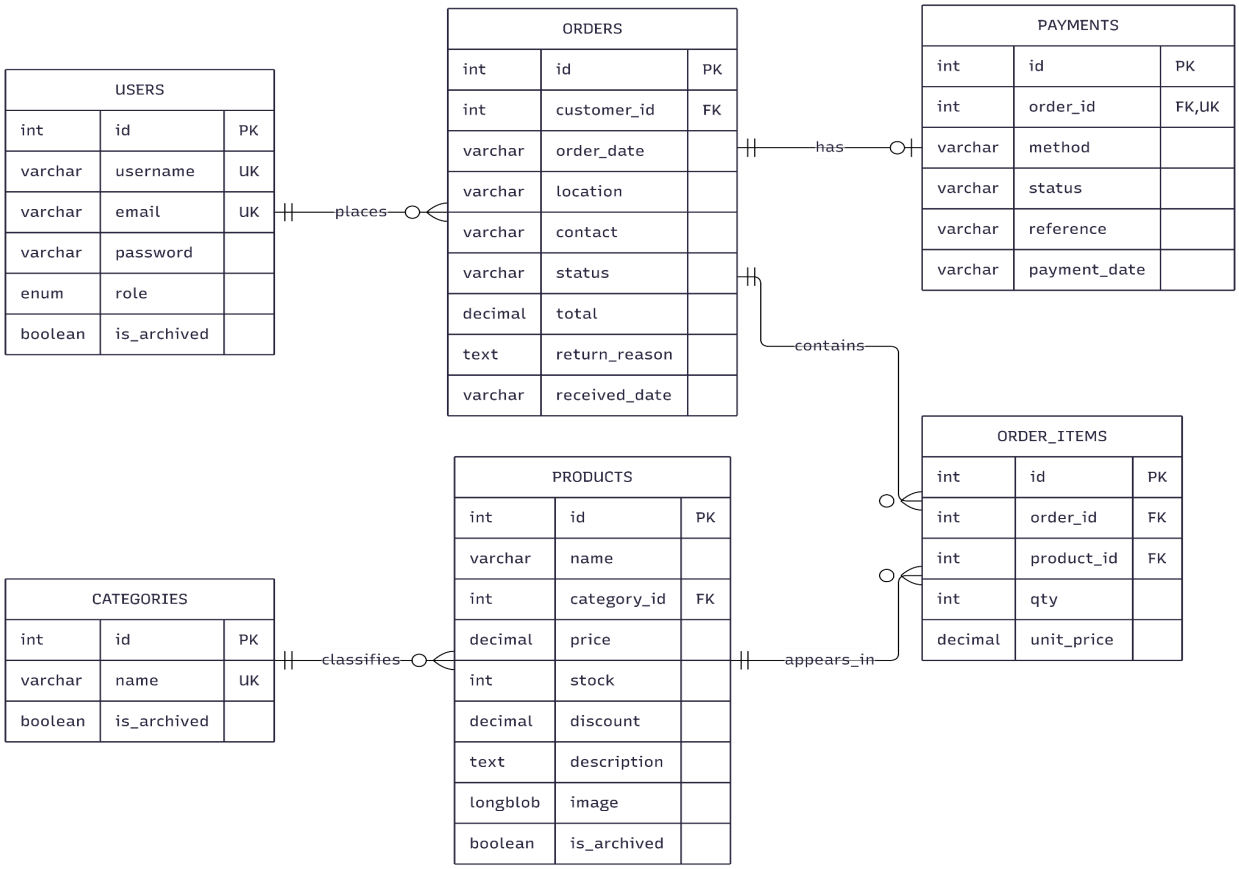


Figure 1. Crow's Foot ERD — Desktop-Based Ordering System for Cherry Cosmetics Shop, showing the six database tables, primary keys, foreign keys, and relationships.

3. Logic Specifications (Step-by-Step Data Flow)

3.1 Feature 1: Processing a Customer Order

- **Customer Selection:** The customer logs in, browses available cosmetic products, selects items, chooses quantities, and adds the selected products to the cart.
- **Cart Calculation:** The application calculates item amounts and the order total, including applicable discounts. It checks product availability and validates the requested quantities.
- **Checkout Details:** The customer provides the required contact and delivery or pickup information and selects an available payment method, if payment-method selection is provided by the current interface.
- **Input Validation:** The application checks required fields, valid quantities, product availability, and any payment reference required by the selected payment method.
- **Order Processing:** The Java application prepares the customer, order, order-item, and payment information for database storage.
- **Database Execution:** The application saves the order and its related records in MySQL. The database operations should use prepared statements for user-supplied values. Related changes, including stock updates, should be performed consistently, preferably within a database transaction when multiple operations must succeed or fail together.
- **Confirmation:** After successful order submission, the application provides appropriate confirmation and displays the order in the customer's order history if that function is implemented. Cart clearing and the initial order status must follow the actual application workflow.

3.2 Feature 2: Archiving a Product Record (Soft Deletion)

- **User Action:** The administrator selects a product and chooses the archive action, if the function is available in the current interface.
- **Record Selection:** The application identifies the selected product using its database ID.
- **Database Update:** The application updates the product's is_archived field rather than permanently deleting the record. A typical convention is 0 for active and 1 for archived.
- **Data Preservation:** The product record remains in the database. Queries for active products should exclude archived records where appropriate.
- **Result:** The administrator can manage active products while preserving historical product information. Related order records should remain intact, and archived products should not appear as available for new orders unless the application explicitly supports that behavior.

4. System Deployment & Setup Guide (Imperative Mood)

Follow these steps to set up and run the Cherry Cosmetics desktop application on a local computer:

- **Install the Environment:** Install a compatible JDK, preferably JDK 21, and NetBeans IDE with Apache Ant support.
- **Start MySQL:** Open the XAMPP Control Panel and start the MySQL service. Make sure MySQL starts successfully before proceeding.
- **Extract the Project:** Extract the Cherry Cosmetics project ZIP file into a suitable folder.
- **Open the Project:** Open NetBeans, select **File** → **Open Project**, and choose the project folder containing build.xml and nbproject.

- **Import the Database:** Open phpMyAdmin and import database/cherry_cosmetics.sql. Confirm that the script creates the cherry_cosmetics database and its six tables.
- **Download the JDBC Driver:** Download MySQL Connector/J from the official MySQL website. Select **Platform Independent** and download the ZIP Archive.
- **Add the Driver:** Extract the downloaded archive. Create the project's lib folder if it does not exist, then place the Connector/J JAR inside it as lib/mysql-connector-j.jar. Ensure the filename matches the actual JAR and that NetBeans includes it in the project classpath.
- **Configure the Connection:** Open src/cherrycosmetics/data/Database.java and ensure that the JDBC URL, username, and password match the local MySQL server. The documented default settings are localhost:3306, database cherry_cosmetics, username root, and a blank password. Change these settings if your local installation uses different credentials.
- **Build the System:** In NetBeans, select **Clean and Build Project** and wait for the build to finish. Resolve any compilation errors before proceeding.
- **Launch the Application:** Run cherrycosmetics.Main, provided it is the configured main class. Import the database schema and confirm the MySQL connection before launching the application.
- **Verify the Features:** Test administrator login, customer registration and login, product browsing, checkout, order management, report filters, CSV export, and eligible PNG receipt generation. Confirm that each feature works and that the corresponding database records are saved correctly.

Default Administrator Credentials

Use the following credentials only if they are configured in the current project:

- **Username:** admin
- **Password:** admin123

5. Core Competencies & Security Matrix

- **Input Validation & SQL Safety:** The application should validate user inputs, required fields, product quantities, and account information. JDBC prepared statements should be used for user-supplied SQL values to help reduce SQL injection risks. Test duplicate usernames, duplicate email addresses, invalid quantities, missing checkout details, and database connection failures against the actual application.
- **Error Handling & Stability:** The application should provide understandable feedback when an operation fails instead of exposing unnecessary technical database details to users. Test database errors, invalid login attempts, incomplete forms, and failed order submissions. A successful compilation does not prove that every function works correctly at runtime.
- **Password Security:** If the current application stores passwords as plain text, password storage is a security weakness. Implement a suitable password-hashing method, such as BCrypt or PBKDF2, before using the application with real accounts. Do not store recoverable plain-text passwords or send existing passwords through a password-reset process. A safer reset workflow should verify the user's authorization and allow a new password to be set securely.
- **Administrator Account Security:** Demonstration credentials should not be treated as production credentials. Change default credentials before deployment, restrict administrator access, and avoid embedding sensitive passwords in source code or public project files.

- **Reporting & Exporting:** Verify report totals, date filters, and exported CSV contents against the actual database records. Ensure that exported files contain only the information necessary for the report and that calculated totals are consistent with the application's order and payment rules.
- **Database Integrity & Archiving:** Verify primary keys, foreign keys, unique constraints, and related-table operations. If soft deletion is used, confirm that archiving a record does not unintentionally remove order history or break database relationships.

6. Final Deliverable Assembly & Workspace Checklist

Project Application Name: Desktop-Based Ordering System for Cherry Cosmetics Shop

Course & Section: Information Management 2 (BSIT)

Step 1: UI Consistency & Source Code Hygiene

☑	Unused testing variables, hardcoded dummy values, and debugging print statements removed.
☑	Fonts, margins, and button formatting match seamlessly across all interface screens.
☑	Raw database errors replaced with custom feedback validation banners.

Verified Application Files: Main.java, MainFrame.java, Login.java, Signup.java, AdminHome.java, AdminProducts.java, AdminOrders.java, AdminCustomers.java, AdminReports.java, CustomerHome.java, Products.java, Cart.java, MyOrders.java, Account.java, Database.java, Models.java, and Store.java.

Step 2: Database Schema Script Generation

☑	All table creations, foreign key constraints, and views exported into a single .sql text file.
☑	Schema script organized in chronological execution order for error-free deployment.
☑	Personal test data removed; seeded with standard academic test records.

Master Database Schema File Name: database/cherry_cosmetics.sql

Active Database File Name: database/cherry_cosmetics.sql

Step 3: Document Package Assembly

☑	Setup guide written strictly in the step-by-step imperative mood with default credentials.
☑	Master documentation cleared of boilerplate placeholder text or empty headings.
☑	Documentation file tree mirrors the active repository directory structure.
☑	Crow's Foot ERD diagram generated from the actual schema and embedded in Section 2.3.

Master Technical Manual File Name: CherryCosmetics_Final_Technical_Documentation.md

STUDENT CONFIRMATION: I verify that the code repository, database schema, and documentation assets checked above match our final production submission package.

Student Signature: _____ Date: _____